22/6/25

HW2 Answers

1) No, the Perceptron doesn't necessarily make the same number of mistakes or end up with the same final weights, if the data order changes.

For example, let's look at perceptron algorithm.

⇒ Input: $(x_1, y_1), \ldots (x_m, y_m)$ where $x_i \in R^d$, $y_i \in \{-1, +1\}$

⇒ Initialize $w = 0$

⇒ For each (x_i, y_i), if $y_i \cdot (w^t x_i) \leq 0$, then

$$w \leftarrow w + y_i x_i$$

Here, the update depends on order of examples, because each mistake updates w & the updated w is used to classify next example.

Counterexample:

Let's consider a 3D dataset & training set S consist of 3 examples.

Example	x	y
A	(1, 0, 1)	+1
B	(1, 1, 0)	-1
C	(-1, 0, -1)	+1

1) Lets run perceptron in order [A, B, C]

Initial weight $w = (0, 0, 0)$

i) $A = (1, 0, 1)$, $y = +1$

$\Rightarrow y \cdot (w \cdot x) = 0 \leq 0 \Rightarrow$ mistake

So, update $w = w + yx = (0,0,0) + 1 \cdot (1, 0, 1)$

$w = (1, 0, 1)$

2) $B = (1, 1, 1)$, $y = -1$

$\Rightarrow y \cdot (w \cdot x) = (-1)(1+0+1) = -2 \leq 0 \Rightarrow$ mistake

So, update $w = w + yx = (1, 0, 1) + (-1) \cdot (1, 1, 1)$

$w = (0, -1, 0)$

3) $C = (-1, 0, -1)$, $y = +1$

$\Rightarrow y \cdot (w \cdot x) = 1 \cdot (0+0+0) = 0 \leq 0 \Rightarrow$ mistake

So update, $w = w + yx = (0, -1, 0) + (-1, 0, -1)$

Final weight $w = (-1, -1, -1)$ // Mistakes = 3 //

2) Now lets run perceptron in order [C, B, A]

Initial weight $w = (0, 0, 0)$

1) $C = (-1, 0, -1)$, $y = +1$

$\Rightarrow y \cdot (w \cdot x) = 0 \leq 0 \Rightarrow$ mistake

So update, $w = w + yx = (0, 0, 0) + (-1, 0, -1)$

$w = (-1, 0, -1)$

2) $B = (1, 1, 1)$, $y = -1$

$\Rightarrow y \cdot (w \cdot x) = (-1) \cdot (-1+0+-1) = 2 > 0 \Rightarrow$ Correct

So don't update $w \Rightarrow w = (-1, 0, -1)$

3) $A = (1, 0, 1)$, $y = +1$

$\Rightarrow y \cdot (w \cdot x) = (-1+0+(-1)) = -2 \leq 0 \Rightarrow$ mistake

So update, $w = w + yx = (-1, 0, -1) + (1, 0, 1)$

Final weight $w = (0, 0, 0)$ //

Mistakes = 2 //

So, if data order changes, the number of mistakes & weights won't be necessarily the same.

---

2) Given,

Loss function $\phi(z) = \max(0, -z)$

For each training example (x_i, y_i) $\{ \phi(y_i \cdot w^T x_i) = \max(0, -y_i \cdot w^T x_i)$

Objective: Minimize average loss over all examples.

$$\frac{1}{m} \sum_{i=1}^{m} \phi(y_i \cdot w^T x_i)$$

SGD update rule, when using random sample (x_i, y_i)

$$\Rightarrow \underset{new}{\omega} \leftarrow \underset{old}{\omega} - \eta \cdot \nabla \phi(y_i \cdot \omega^T x_i)$$

Step1: Complete Gradient of $\phi(z)$:

$$\phi(z) = \begin{cases} -z & \text{if } z < 0 \\ 0 & \text{if } z \geq 0 \end{cases}$$

So, derivative $\phi'(z)$,

~~$\phi'(z) = \begin{cases} -1 & \text{if } z < 0 \\ 0 & \text{if } z \geq 0 \end{cases}$~~

$$\phi'(z) = \begin{cases} -1 & \text{if } z < 0 \\ 0 & \text{if } z > 0 \end{cases} \quad \text{(ignore } z=0 \text{ as given)}$$

We know, $z = y_i \cdot \omega^T x_i$

$$\Rightarrow \nabla_\omega \phi(y_i \cdot \omega^T x_i) = \begin{cases} -y_i x_i & \text{if } y_i \omega^T x_i < 0 \\ 0 & \text{if } y_i \omega^T x_i > 0 \end{cases}$$

Step2: Adding that into SGD update

Using, $\omega \leftarrow \omega - \eta \cdot \nabla_\omega \phi(y_i \omega^T x_i)$

So,

$$\omega \leftarrow \begin{cases} \omega + \eta y_i x_i, & \text{if } y_i \omega^T x_i < 0 \\ \omega & \text{otherwise} \end{cases}$$

→ This is the same as Perceptron rule except the learning rate η

Insights:

→ The Perceptron algorithm updates the weights as,

$$\omega \leftarrow \omega + y_i x_i \quad \text{if } y_i \omega^T x_i \leq 0$$

→ SGD with loss $\phi(z) = \max(0, -z)$, performs the same kind of update only when the prediction is wrong (i.e., margin is negative), it moves the weight in the direction of $y_i x_i$, scaled by learning rate η.

— X —

3) Given,

Let the domain be R & C denotes the concept class of 3-piece classifiers defined as,

$$h_{\theta_1,\theta_2,b}(x) = \begin{cases} b & \text{if } x \in [\theta_1, \theta_2] \\ -b & \text{otherwise} \end{cases}$$

where $\theta_1 < \theta_2$ & $b \in \{-1, 1\}$

Let H denote the class of decision stumps,

$$h_{\theta,b}(x) = \begin{cases} b & \text{if } x \leq \theta \\ -b & \text{otherwise} \end{cases}$$

a) Existence of Low-Error Decision Stump:

The function c(x) breaks real line into 3 contiguous regions.

1) Left $\Rightarrow x < \theta_1 \rightarrow$ label: $-b$

2) Middle $\Rightarrow x \in [\theta_1, \theta_2] \rightarrow$ label: b

3) Right $\Rightarrow x > \theta_2 \rightarrow$ label: $-b$

Now the probability mass function in these 3 regions.

$\Rightarrow p_L = P_r[x < \theta_1]$
$\Rightarrow p_m = P_r[\theta_1 \leq x \leq \theta_2]$
$\Rightarrow p_R = P_r[x > \theta_2]$

As total probability is 1,

$$p_L + p_m + p_R = 1$$

So, if let's say 2 of these regions are greater than $1/3$, then atleast the third region will be less than $1/3$.

$\Rightarrow$ This proves, for any distribution & any $c \in C$, there exists stump $h \in H$,

$\Rightarrow P_r[h(x) \neq c(x)] \leq \frac{1}{3}$ //

b) Empirical risk minimizer

Given, $S = \{(x_1, y_1) \ldots (x_m, y_m)\}$
where $x_i \in R$, $y_i \in \{-1, +1\}$

To find $h_{\theta,b} \in H$, minimizing the training error.

Algorithm:

-> Sort examples by input values x_i.

-> Consider midpoints between consequtive examples as thresholds θ. Also consider one above the smallest & one above the largest.

So, for sorted $x_1 \leq x_2 \leq \dots \leq x_m$,

$$\theta_k = \frac{x_k + x_{k+1}}{2}, \text{ for } k = 1 \text{ to } (m-1)$$

-> For each threshold, try both labels $b=+1$ & $b=-1$

-> For each (θ_k, b), compute classification predictions

$$h_{\theta_k, b}(x) = \begin{cases} b & x \leq \theta_k \\ -b & x > \theta_k \end{cases}$$

-> Compute the training error on the dataset.

-> Pick a stump with the lowest training error.

-> As it is sorted, the complexity is $O(m \log m)$ & so this procedure is efficient & yields ERM over H.

c) Why we can expect generalization from training to true error.

1) Complexity Reduction from C to H:

-> C is more complex than H as C allows more intricate decision boundaries, while H restricts them to simple threshold based decisions.

-> So, we reduce hypothesis space's capacity which helps in reducing overfitting.

2) VC Dimension:

-> VC Dimension of H is lower than C.

-> Lower VC implies fewer examples are needed to shatter hypothesis class.

-> So model will generalize for an limited number of training samples.

3) Sample Complexity:

→ With reduced hypothesis space, the sample complexity decreases.

→ Thus, choosing 'm' sufficiently large allows the training error to be a reliable estimate of the true error.

⇒ Thus, by learning in H, we can ensure that the training performance reflects true performance, enabling us to weakly learn C using H.

---

4) In AdaBoost, after fitting a weak classifier h_t, & computing its error ϵ_t, the distribution D_{t+1} is updated such that, the misclassified examples receive higher weights.

To prove,

The accuracy of h_t on this new distribution D_{t+1} is exactly $1/2$.

Proof:

We know that, when we update the distribution

$$D_{t+1}(i) = \frac{D_t(i) \cdot \exp(-\alpha_t y_i h_t(x_i))}{Z_t}$$

where

$$\alpha_t = \frac{1}{2} \ln\left(\frac{1-\epsilon_t}{\epsilon_t}\right)$$

Now, Accuracy Calculation

$$\sum_{i=1}^{m} D_{t+1}(i) \cdot I[h_t(x_i) = y_i]$$

$$= \frac{1}{Z_t} \sum_{i=1}^{m} D_t(i) \cdot e^{(-\alpha_t y_i h_t(x_i))} \cdot I(h_t(x_i) = y_i)$$

This can be split into 2 parts,

based on whether $h_t(x_i) = y_i$

(or)

$h_t(x_i) \neq y_i$

$$\Rightarrow \frac{1}{Z_t}\left[\sum_{i:h_t(x_i)=y_i} D_t(i) \cdot e^{(-\alpha_t)} + \sum_{i:h_t(x_i)\neq y_i} D_t(i) \cdot e^{(\alpha_t)}\right]$$

Thus the new distribution D_{t+1}, the classifier h_t has accuracy,

$$Pr[h_t(x_i) = y_i] = Pr[h_t(x_i) \neq y_i] = \frac{1}{2}$$

=> This holds true because, the update rule exponentially increases the weight of misclassified examples & decreases weight of correctly classified ones, thus resulting in balancing the distribution between the 2 equally.

=> This forces AdaBoost to focus on harder examples in next iterations.